

Attachment Architecture

Last updated: 2026-05-11

ArtHound surfaces attachments (images, video, PDFs, documents) from source tools inline in the UI, viewable in-browser, not just downloadable. The system is designed around two principles:

1. **User-intent-driven copying.** No proactive crawling. A file is copied to ArtHound's storage only when a user explicitly triggers it, either by dispatching an asset to a vendor, or by opening an attachment in the Asset Viewer.
 2. **Content-addressed storage.** Every file is stored at `attachments/sha256/{hex_hash}` in Supabase Storage. Identical files across multiple dispatches or studios share one blob. No path-sanitization issues. Dedup is free.
-

Storage Layer

Bucket: `attachments` (Supabase Storage, service-role access only)

Path scheme: `sha256/{64-char-hex-hash}`

The SHA-256 hash is computed over the raw file bytes at copy time. This makes the storage path deterministic and collision-safe. It also means the existence check before upload (`HEAD /object/ ...`) is sufficient to avoid re-uploading identical content.

`attachment_refs` table is the reverse index, every `(content_hash, dispatch_id)` pair is recorded so the purge routine knows which blobs have active dispatch references.

Copy Triggers

1. Copy-on-dispatch (vendor path)

When a studio dispatches an asset payload to a vendor, the dispatch endpoint immediately enqueues an `attachment_copy_jobs` row. A background worker drains the queue and downloads each attachment from the source tool, hashes it, uploads to Supabase Storage, and patches `content_hash` back into `payload_dispatches.payload_data` for each attachment item.

This ensures the vendor always sees what was sent at dispatch time, the snapshot is frozen. If the studio later modifies or deletes the source file, the vendor's copy is unaffected.

2. Copy-on-first-view (studio path)

When a studio user opens an attachment in the Asset Viewer, the proxy endpoint:

1. Checks `replicated_assets.meta[field_key][idx].content_hash`
2. **Fast path** (subsequent views): `content_hash` present → stream directly from Supabase Storage. No source credentials touched.
3. **First-view path**: download from source (Airtable/Jira) with live credentials → hash → upload to Storage if not already present → patch `content_hash` and resolved `mimetype` back into `replicated_assets.meta` → return the already-downloaded bytes to the browser.

The user accepts a one-time loading delay for large files. All subsequent views are served from Storage.

If the source record changes (sync detects a different `source_hash`), the entire `meta` is rewritten from source, `content_hash` is lost, and the next view re-copies. This is correct: the file may have changed.

Job Queue

Table: `attachment_copy_jobs`

Column	Type	Notes
<code>id</code>	UUID	PK
<code>dispatch_id</code>	UUID	FK → <code>payload_dispatches(id)</code> ON DELETE CASCADE
<code>status</code>	TEXT	<code>pending</code> / <code>processing</code> / <code>done</code> / <code>failed</code>
<code>attempts</code>	INT	Incremented on each claim
<code>last_error</code>	TEXT	Last failure message (truncated to 1000 chars)
<code>created_at</code>	TIMESTAMPTZ	
<code>updated_at</code>	TIMESTAMPTZ	

Claim pattern: Atomic optimistic claim, `PATCH ... WHERE status='pending'`. If another worker already claimed the job, the PATCH touches 0 rows and the job is skipped safely. Supports a single-worker model with no locking required.

Retry: Up to 3 attempts. On exhaustion, status is set to `failed` with `last_error` populated.

Worker: `drain_attachment_jobs()` in `lib/attachments.py`, called by `_attachment_drain_loop()` in `main.py` every 30 seconds. This loop runs unconditionally, it does not depend on `SYNC_POLL_INTERVAL_SECONDS`.

DB Schema

```
CREATE TABLE attachment_copy_jobs (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  dispatch_id UUID          NOT NULL REFERENCES payload_dispatches(id) ON DELETE CASCADE,  
  status      TEXT          NOT NULL DEFAULT 'pending'  
  CHECK (status IN ('pending', 'processing', 'done', 'failed')),  
  attempts    INT           NOT NULL DEFAULT 0,  
  last_error  TEXT,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX attachment_copy_jobs_status_created ON attachment_copy_jobs(status, created_at);  
  
CREATE TABLE attachment_refs (  
  content_hash TEXT NOT NULL,  
  dispatch_id  UUID NOT NULL REFERENCES payload_dispatches(id) ON DELETE CASCADE,  
  PRIMARY KEY (content_hash, dispatch_id)  
);  
CREATE INDEX attachment_refs_dispatch ON attachment_refs(dispatch_id);
```

Backend Components

lib/attachments.py

Core logic. All Supabase Storage interaction lives here.

Function	Purpose
<code>_storage_exists(hash, client)</code>	HEAD check, returns bool
<code>_storage_upload(hash, data, content_type, client)</code>	POST upload, raises on failure
<code>_get_studio_source_creds(studio_id, client)</code>	Fetches and decrypts credentials; refreshes Jira OAuth tokens
<code>_source_fetch_headers(source_type, creds)</code>	Returns auth headers for fetching from source
<code>_find_attachment_items(data)</code>	Walks <code>payload_data['data']</code> and returns <code>[(field_key, idx, item)]</code> for all attachment lists
<code>copy_payload_attachments(dispatch_id)</code>	Full copy pipeline for one dispatch, downloads, hashes, uploads, patches <code>payload_data</code>
<code>enqueue_attachment_copy(dispatch_id)</code>	Inserts a <code>pending</code> job row
<code>drain_attachment_jobs()</code>	Claims and processes one pending job per call
<code>purge_orphaned_attachments()</code>	Deletes blobs with no active dispatch reference

routes/attachments.py

Two proxy endpoints plus an admin trigger.

GET /api/attachments/asset/{canonical_asset_id}/{field_key}/{idx}

Studio-only. Copy-on-first-view from source tool.

- Checks `replicated_assets.meta[field_key][idx].content_hash`
- If present: streams from Supabase Storage (fast path)
- If absent: downloads from source with studio credentials → hashes → uploads → patches meta → returns bytes

GET /api/attachments/payload/{dispatch_id}/{canonical_asset_id}/{field_key}/{idx}

Studio or vendor. Serves frozen copy from Supabase Storage.

Auth predicate (`_authorize_payload_attachment`) checks in order: 1. Dispatch row exists → 404 if not (no existence leakage) 2. Caller is the vendor on this dispatch, or a studio member of the dispatching studio → 404 if not 3. Dispatch has not been revoked → 403 with reason 4. `canonical_asset_id` is present in `payload_data.assets` → 404 if not (prevents cross-dispatch ID guessing)

Returns `202` if `content_hash` is null (copy job not yet complete). The frontend handles this as a "pending" state.

POST /api/attachments/admin/purge

Admin-only. Manually triggers `purge_orphaned_attachments()`. Returns `{deleted, kept, errors}`.

Background Loops (`main.py`)

Loop	Interval	Purpose
<code>_attachment_drain_loop</code>	30s, always-on	Drains <code>attachment_copy_jobs</code> queue
<code>_attachment_purge_loop</code>	24h (configurable via <code>PURGE_ATTACHMENTS_INTERVAL_HOURS</code>)	Deletes orphaned storage blobs

Both are independent `asyncio.Task` s cancelled on shutdown. The drain loop does not depend on sync polling being enabled.

Purge Routine

`purge_orphaned_attachments()` runs nightly:

1. Fetches all non-revoked dispatch IDs from `payload_dispatches`

2. Fetches all `attachment_refs` rows and builds the keep-set: content hashes with at least one active dispatch reference
3. Lists all objects in the `attachments` bucket (paginated, 1000 per page)
4. Deletes any blob whose hash is not in the keep-set, in batches of 100

This removes: - Studio-side cached copies (no dispatch ref), re-copied on next view - Copies from revoked dispatches, vendor can no longer access them anyway

Set `PURGE_ATTACHMENTS_INTERVAL_HOURS=0` to disable the nightly loop. The admin button in the Account modal triggers the same routine on demand.

JWT / Browser Compatibility

Browser `<img src>`, `<video src>`, `<a download>`, and pdf.js internal fetch cannot send custom `Authorization` headers. All proxy endpoints require a Bearer JWT.

Solution: The `useMediaUrl(proxyUrl)` hook fetches via `apiFetchRaw` (which injects the JWT), then creates a local blob URL via `URL.createObjectURL()`. All media components receive this blob URL, the browser never makes a credentialed request directly.

On unmount or `proxyUrl` change, the hook revokes the blob URL to avoid memory leaks.

```
Browser → useMediaUrl → apiFetchRaw (JWT) → proxy endpoint → Supabase Storage
      ↓
      URL.createObjectURL()
      ↓
      <img src={blobUrl}> / <video src={blobUrl}> / pdf.js(blobUrl)
```

A `202` response from the proxy is interpreted as `pending: true`, all viewers show a "still being processed" state rather than an error.

Frontend Components

`frontend/src/hooks/useMediaUrl.js`

```
const { blobUrl, loading, error, pending } = useMediaUrl(proxyUrl)
```

Returns: - `blobUrl`, object URL ready for use in `src` attributes or pdf.js - `loading`, fetch in progress - `error`, fetch failed (non-2xx, non-202) - `pending`, server returned 202 (copy job not yet complete)

frontend/src/components/media/

Component	Purpose
AttachmentGallery.jsx	Container for a field's attachment list; manages lightbox open/close state
AttachmentCard.jsx	Thumbnail card; image blob preview for images, type icon for others; clock icon when <code>proxyUrl</code> is null
MediaLightbox.jsx	Full-screen overlay; keyboard nav (Escape, arrows); routes to correct viewer via <code>viewerType()</code>
ImageViewer.jsx	<code></code> with click-to-zoom; pending / loading / error states
VideoViewer.jsx	<code><video controls></code> ; pending / loading / error states
PdfViewer.jsx	pdf.js canvas renderer; page nav and zoom controls; pending / loading / error states
DocumentCard.jsx	Fallback for unsupported types; programmatic download via <code>apiFetchRaw</code> + blob
mediaUtils.js	<code>resolveMimeType(mimetype, filename)</code> , extension fallback map; <code>viewerType()</code> , returns <code>'image' / 'video' / 'pdf' / 'document'</code>

Adding a new media type: add a viewer component, add the MIME type(s) to `mediaUtils.js`, add one case to `MediaLightbox`.

frontend/src/lib/fields.js

`formatRawFields(rawFields, proxyUrlFn)` detects attachment arrays (lists of objects with a `url` key) and emits a single `{ type: 'attachments', label, items[] }` entry instead of individual link entries. Each item carries `{ filename, mimetype, size_bytes, proxyUrl }`.

`proxyUrlFn` is a `(fieldKey, idx) ⇒ string` callback that generates the correct proxy URL for the current context (asset viewer vs. vendor inbox).

frontend/src/lib/api.js

Helper	Returns
<code>apiFetchRaw(path, opts)</code>	Raw <code>Response</code> with JWT injected
<code>assetAttachmentUrl(canonicalAssetId, fieldKey, idx)</code>	<code>/api/attachments/asset/ ...</code> URL
<code>payloadAttachmentUrl(dispatchId, canonicalAssetId, fieldKey, idx)</code>	<code>/api/attachments/payload/ ...</code> URL

Source Tool Integration

Airtable

The Airtable adapter (`lib/connectors/adapters/airtable.py`) deserializes `attachments` fields into `[AttachmentValue(url, filename, mimetype, size_bytes)]` . URLs are Airtable pre-signed S3 URLs that expire in ~2 hours.

Because URLs expire, the copy-on-first-view pattern re-fetches from source using live credentials on expiry. The first view after expiry incurs a download delay; subsequent views in the same window are served from Storage.

No auth headers are needed for the download step, pre-signed URLs carry auth in query params.

Jira

Jira attachment fields have schema `{type: "array", items: "attachment"}` . The field type stored in `field_type_map` is `"array"` , which routes through `_deserialize_array` → detects `"filename"` key → calls `_deserialize_attachment` . The Jira adapter maps `raw.get("content")` to `AttachmentValue.url` and `raw.get("mimeType")` to `AttachmentValue.mimetype` .

Jira attachment URLs

(`https://api.atlassian.com/ex/jira/{cloudId}/rest/api/3/attachment/content/{id}`) are stable but require a valid OAuth access token on every request. `_get_studio_source_creds` calls `get_jira_token` to refresh before download.

After first-view copy, subsequent serves come from Storage with no Jira dependency.

Integration Points

Surface	Endpoint	proxyUrlFn source
Asset Viewer (Attachments tab)	<code>/api/attachments/asset/ ...</code>	<code>assetAttachmentUrl</code> in <code>AttachmentsTab.jsx</code>
Reviews tab (attachment context)	<code>/api/attachments/asset/ ...</code>	<code>assetAttachmentUrl</code> in <code>ReviewsTab.jsx</code>
Vendor Inbox (payload modal)	<code>/api/attachments/payload/ ...</code>	<code>payloadAttachmentUrl</code> in <code>VendorInbox.jsx</code>

All three surfaces use the same `AttachmentGallery` → `MediaLightbox` → viewer chain.

Open Design Questions

The following were deferred intentionally and require a design session before implementation:

- **Stale studio-side copies.** If a source attachment is replaced, the cached Storage copy becomes stale. Should ArtHound detect this and invalidate, or offer a "live view" toggle (analogous to the ArtHound vs. Source switcher on Work)?
- **expires_at enforcement.** Dispatch expiry is not currently checked in the proxy or purge. Expired dispatches still serve attachments. Decide whether expiry should behave like revocation.
- **Per-studio storage quotas.** No storage reporting exists. At scale, copy-on-first-view could accumulate significant storage per studio.
- **Audit trail on purge.** Purged blobs are permanently deleted. Consider a soft-delete or grace period if audit requirements emerge.
- **Worker scaling.** The current single-worker drain is safe but not scalable. If multiple workers are needed, switch the claim query to `SELECT ... FOR UPDATE SKIP LOCKED`.